Implementation and Evaluation of the Kaminsky Attack

A Theoretical Overview and Step-by-Step Practical Tutorial in SEED Labs

Emre Topcu
Computer Science & Engineering
Akdeniz University
Antalya, Turkiye
20220808011@ogr.akdeniz.edu.tr

Muhammed Kaan Uman
Computer Science & Engineering
Akdeniz University
Antalya, Turkiye
20220808603@ogr.akdeniz.edu.tr

***Abstract*—The Domain Name System (DNS) cache poisoning attack is a critical threat that misdirects users to malicious servers by corrupting the resolver's cache. Traditional poisoning attacks are hindered by the caching effect, which forces an attacker to wait for a cached record to expire before attempting another spoof [1]. Dan Kaminsky's discovery elegantly bypasses this limitation by continuously querying randomly generated subdomains, allowing for rapid, continuous exploitation [3]. This paper provides both a deep theoretical analysis of the Kaminsky attack and a comprehensive, step-by-step tutorial of its implementation. Conducted within the SEED Labs environment, we detail the virtualization setup, reconnaissance, preliminary Scapy packet manipulation, hybrid payload crafting, and the precise multi-terminal execution required to win the transaction ID race condition.**

***Index Terms*—DNS, Cache Poisoning, Kaminsky Attack, Scapy, Network Security, SEED Labs.**

I. Introduction to DNS Cache Poisoning

DNS serves as the phonebook of the Internet, translating human-readable domain names into IP addresses. When a local DNS server resolves a query, it caches the result to improve future response times. If an attacker can forge a response from the authoritative nameserver and ensure it arrives before the legitimate response, the local DNS server will cache the forged data, a process known as DNS Cache Poisoning.

Because the attacker in a remote setting cannot sniff the packets to determine the 16-bit transaction ID, they must guess it [1]. However, if they guess incorrectly and the legitimate response arrives first, the authentic record is cached, and the attacker is locked out until the Time-To-Live (TTL) expires. The Kaminsky attack defeats this caching effect by querying non-existent subdomains (e.g., `aaaaa.example.com`). Since the subdomain is not in the cache, the local server is forced to query the authoritative server every time, giving the attacker unlimited attempts to flood the server with spoofed responses and guess the transaction ID [3].

II. Phase 1: Experimental Environment Setup

To safely demonstrate this attack, we utilized the SEED Ubuntu 20.04 Virtual Machine (VM). The environment relies on Docker containers to simulate an isolated, real-world Local Area Network (LAN) [2].

A. Container Initialization and Management

Upon booting the SEED Ubuntu VM, we opened the built-in web browser, navigated to the SEED Labs portal, and downloaded the required lab archive. We extracted the `Labsetup.zip` file into the local `Downloads` directory. Navigating to the `Downloads/Labsetup` folder via the terminal, we built and initialized the network using the Docker Compose aliases provided by the SEED environment:

```
$ cd Downloads/Labsetup
$ dcbuild  # Builds the container images
$ dcup     # Starts the containers in the background
```

Once the containers were successfully deployed, we needed a way to identify their unique Docker IDs to open interactive shell sessions for our multi-terminal attack execution. We utilized the `dockps` alias to list the active containers and their roles, and the `docksh <id>` command to spawn a root shell inside a specific container [2].

The following snippet from our terminal demonstrates how we identified the containers and accessed the Local DNS Server:

```
$ dockps
c34b9a0c72d3  seed-attacker
8a1ed19251db  attacker-ns-10.9.0.153
7c3a032c2458  user-10.9.0.5
77a434227c4e  local-dns-server-10.9.0.53

$ docksh 77a
root@77a434227c4e:/#
```

This management workflow allowed us to seamlessly transition between the four primary entities residing on the `10.9.0.0/24` network during the experiment [2]:

- **Attacker (10.9.0.1):** Operates in host mode, granting full access to network interfaces for packet manipulation.
- **User (10.9.0.5):** The victim machine used to trigger recursive DNS queries.
- **Local DNS Server (10.9.0.53):** The target resolver running BIND 9, with DNSSEC disabled.

- **Attacker's Nameserver (10.9.0.153):** The malicious server hosting the `attacker32.com` zone.

Figure 1. Initialization of the SEED Lab Docker containers and identifying their respective IP addresses using the `dockps` command.

III. Phase 2: Reconnaissance and Target Analysis

In a remote cache poisoning attack, the spoofed UDP packet must perfectly mimic the legitimate authoritative nameserver. The target DNS server will reject any response whose source IP does not match the IP it queried [1].

We connected to the Local DNS container using `docksh` and utilized the `dig` command to determine the actual authoritative nameserver for `example.com`.

```
$ dig @10.9.0.53 example.com NS
```

The query revealed that the local DNS server forwarded requests to Cloudflare's nameservers (e.g., `108.162.192.162`). This discovery was critical; hardcoding a generic or outdated IP in our payload would render the attack completely ineffective.

IV. Phase 3: Preliminary Packet Manipulation (Scapy)

Before attempting the high-speed race condition, we isolated the packet crafting process to verify our ability to manipulate DNS traffic using Python's Scapy library [2]. This phase was executed in the shared `/volumes` directory.

A. Constructing the DNS Request

To trigger the local DNS server to query the authoritative nameserver, we crafted a custom DNS query script (`dns_request.py`). This script manually structures the Question Section for a random subdomain (`abcde.example.com`) and targets the local DNS server (`10.9.0.53`). Capturing this execution via Wireshark on the host VM confirmed that the custom packet successfully triggered a corresponding outgoing query from the local DNS server to the Internet.

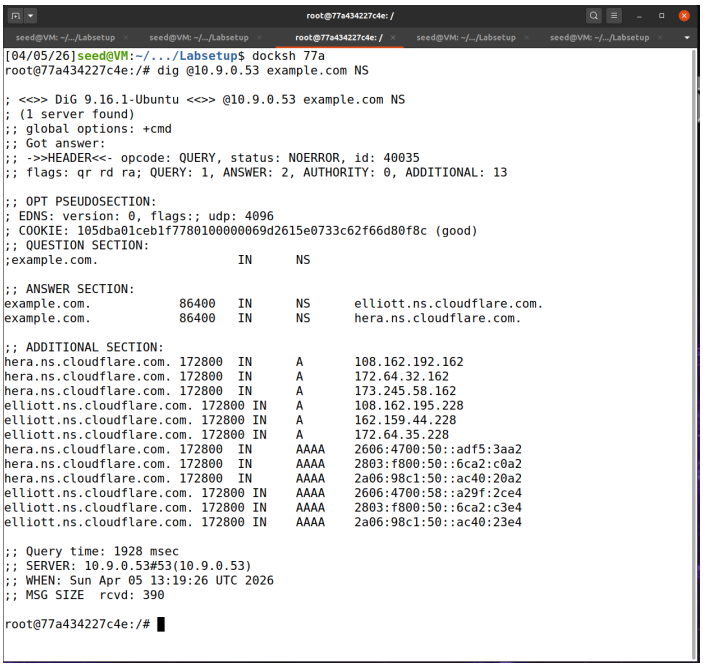

Figure 2. Reconnaissance using the `dig` command to identify the legitimate Nameserver IP addresses in the Authority and Additional sections.

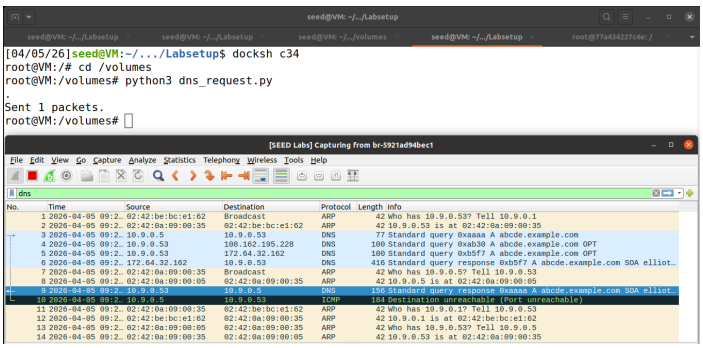

Figure 3. Wireshark verification of the manually crafted Scapy DNS request, capturing the packet generated by `dns_request.py`.

B. Spoofing a Single DNS Reply

Next, we developed `dns_reply.py` to construct a forged response. This script builds a DNS packet where the Answer Section maps the queried subdomain to a fake IP (`1.2.3.4`), and crucially, the Authority Section redirects `example.com` nameserver traffic to `ns.attacker32.com`. Executing this script and analyzing it in Wireshark proved our payload structure was valid, establishing the blueprint for the impending Kaminsky attack.

V. Phase 4: The Hybrid Payload Strategy

Constructing complex DNS packets natively in C is a highly tedious and error-prone task. Conversely, using Python's Scapy library is exceptionally convenient but its packet generation speed is too slow to win a microsecond-level race condition [3]. Therefore, we adopted a hybrid methodology [2].

A. Template Crafting via Shared Volumes (Python)

We consolidated our preliminary Scapy knowledge into a new script (`generate_bin.py`). Instead of sending packets, this script serializes the forged query and the forged response (with the exact Cloudflare IP identified in Phase 2) into raw binary files: `ip_req.bin` and `ip_resp.bin`. The spoofed payload was meticulously constructed to include:

- **Question Section:** A query for a random 5-character subdomain.
- **Answer Section:** A fake IP resolution for the queried subdomain.
- **Authority Section:** An NS record explicitly replacing `example.com`'s nameserver with `ns.attacker32.com`.
- **IP Layer:** The source IP was explicitly forged as the Cloudflare IP discovered in Phase 2.
- **UDP Layer (Port 33333):** We hardcoded the destination port to 33333. In real-world modern DNS servers, Source Port Randomization (SPR) is deployed as a primary defense against Kaminsky attacks. However, the vulnerable SEED environment intentionally disables SPR, fixing the query source port at 33333, which reduces the attacker's required guessing entropy strictly to the 16-bit transaction ID [2].

B. High-Speed Flooding Configuration (C)

With the binary templates generated, we turned to the provided `attack.c` file. Operating inside the attacker container, the C program loads the binaries into memory. During execution, it rapidly iterates through a loop where it overwrites the 5-character random string at precise memory offsets (41 and 64) and increments the 16-bit transaction ID located at offset 28. We increased the inner loop threshold to 3000 packets per trigger to maximize collision probability against modern network interfaces. We then compiled the payload:

```
$ gcc -o attack attack.c
```

VI. Phase 5: Multi-Terminal Attack Execution

The attack's success hinges entirely on precise orchestration. We divided our workspace into three concurrent terminal sessions, executing the steps in strict chronological order to trigger and win the race condition.

A. Step 1: Cache Purge (Terminal 1 - Local DNS)

To ensure the target resolver does not rely on previously cached, authentic records, we flushed its cache manually [2]. This forces the server to initiate a fresh recursive query:

```
$ rndc flush
```

B. Step 2: Payload Flooding (Terminal 2 - Attacker)

Inside the attacker container (`c34`), we executed the compiled C program. This program continuously injected thousands of spoofed UDP packets per second into the network, blindly answering for the random subdomains.

```
$ ./attack
```

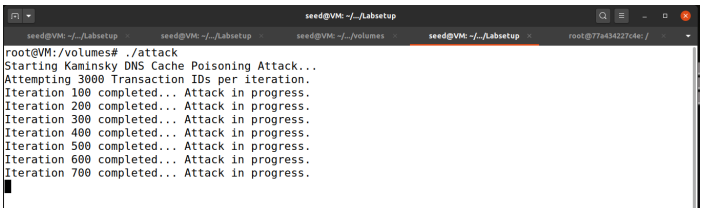

Figure 4. Execution of the optimized C attack program, demonstrating continuous iteration and high-volume spoofed UDP packet flooding.

C. Step 3: Query Trigger (Terminal 3 - User)

With the flood active, we switched to the User container (`7c3`) and issued the catalyst query. This prompts the local DNS server to query the internet, opening the vulnerability window.

```
$ dig @10.9.0.53 www.example.com
```

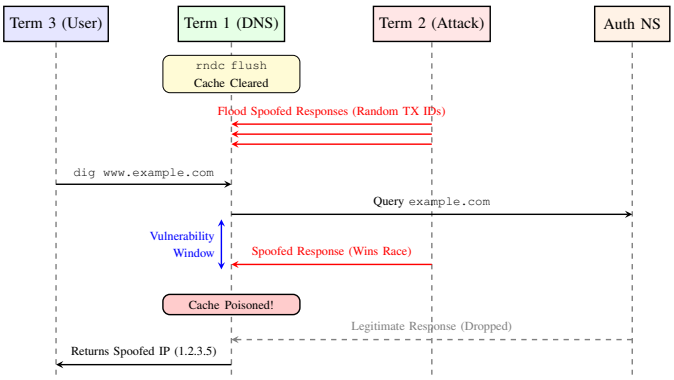

Figure 5. UML Sequence illustrating the race condition orchestrated across the three active terminal windows.

D. Troubleshooting: The Race Condition Dynamics

Because this attack relies on a probabilistic race condition against legitimate nameservers, success on the very first attempt is not guaranteed. It is highly probable that during the initial attempts, the legitimate Cloudflare response might beat the spoofed flood and successfully cache the authentic record. If the `dig` command returns the real IP instead of the spoofed `1.2.3.5`, it indicates a lost race. The attacker must simply reset the vulnerability window by repeating the exact sequence: flush the cache (Terminal 1), sustain the packet flood (Terminal 2), and issue a new trigger query (Terminal 3).

VII. Phase 6: Verification of Cache Poisoning

The attack's efficacy was verified using two distinct methods immediately following a successful trigger execution.

First, we examined the response in the user terminal. The `dig` command returned `1.2.3.5`, which is the exact IP hardcoded in our Scapy template. Seeing this instead of the authentic Cloudflare IP verified that the resolver accepted our spoofed payload and successfully updated its cache.

The ultimate objective of the Kaminsky attack is to hijack the entire domain's authority. To verify this profound impact,

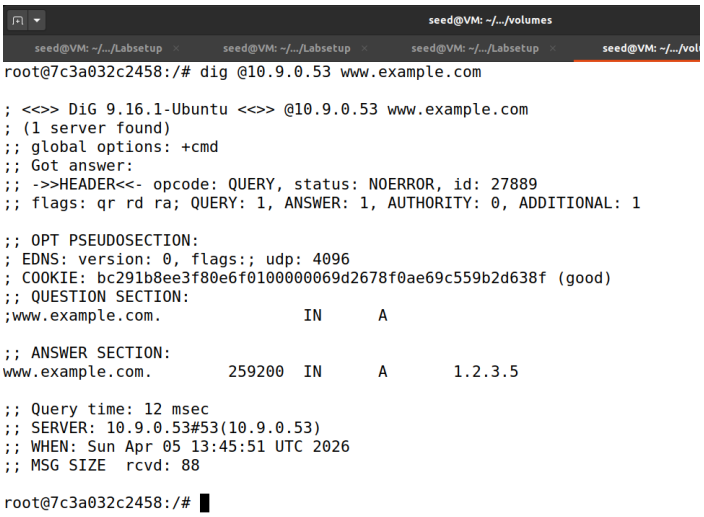

Figure 6. Successful cache poisoning verification: The user machine receives the spoofed IP address (`1.2.3.5`) for the targeted domain.

we returned to the Local DNS terminal and dumped the BIND cache into a database file for inspection:

```
$ rndc dumpdb -cache
$ grep -E "example.com.*NS.*ns.attacker32.com" /var/cache
    /bind/dump.db
```

The output explicitly demonstrated that `example.com` was now assigned to `ns.attacker32.com` with a Time-To-Live (TTL) of 777322 seconds (approximately 9 days). All subsequent traffic for any subdomain of `example.com` would now be routed to the attacker's infrastructure.

Figure 7. Deep verification of the BIND cache: The `grep` output confirms the successful injection of the malicious `ns.attacker32.com` record.

VIII. Conclusion

This paper demonstrated the severe implications of the Kaminsky DNS Cache Poisoning attack, bridging theoretical vulnerabilities with a concrete, laboratory-based execution. By overcoming the traditional limitations of the DNS caching effect through continuous random-subdomain querying, we successfully hijacked the authority of an entire target domain. The methodology highlighted the necessity of a hybrid development approach; leveraging Python's Scapy for precise payload serialization and C for high-speed packet flooding proved critical in winning the microsecond-level transaction ID race condition. Ultimately, this implementation underscores the inherent dangers of UDP's lack of source authentication and the critical need for robust security extensions like DNSSEC. The complete source code and payload templates utilized in this research are publicly available at GitHub Repository

References

[1] W. Du, *Computer & Internet Security: A Hands-on Approach*, 2nd ed. Self-publishing, 2019.
[2] W. Du, "SEED Labs - Remote DNS Cache Poisoning Attack Lab," Syracuse University.
[3] D. Kaminsky, "Black Ops 2008: It's the End of the Cache as We Know It," Black Hat USA, 2008.